Trabajo Práctico Integrador

Comprensión visual de escenas

Primavera 2026

Introducción

El trabajo práctico consiste en desarrollar un sistema de **comprensión visual de escenas** capaz de transformar una imagen en una representación estructurada, describirla y responder preguntas sobre su contenido.

La progresión conceptual del trabajo es:



El sistema deberá localizar objetos mediante un detector *open-vocabulary*, segmentar cada instancia utilizando SAM2 y construir una descripción estructurada en formato JSON que pueda consultarse de manera determinista. Para alcanzar el nivel de promoción deberá incorporar además un modelo de lenguaje liviano para expresar y consultar la información obtenida.

El objetivo principal no es maximizar una única métrica ni encadenar modelos preentrenados. Se evaluará especialmente la capacidad del grupo para:

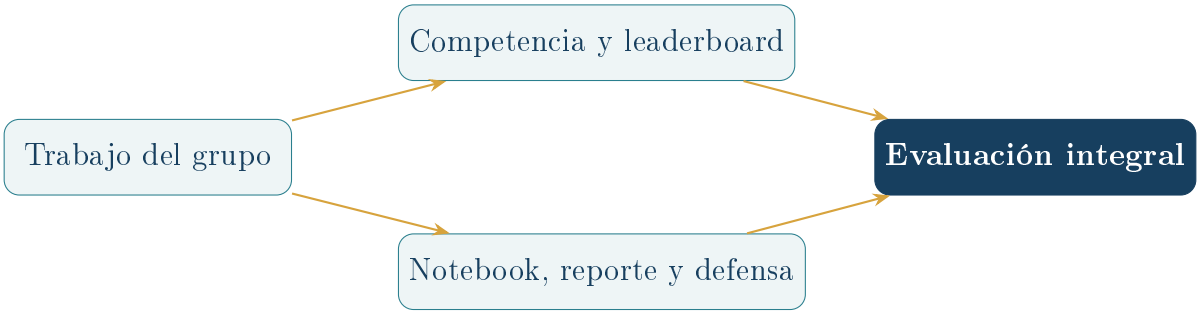
- Diseñar una pipeline modular y trazable.
- Formular hipótesis y realizar experimentos controlados.
- Evaluar cada etapa con métricas apropiadas.
- Analizar errores y su propagación a lo largo del sistema.
- Distinguir información observada, derivada y desconocida.
- Justificar las decisiones técnicas adoptadas.

No se busca resolver el trabajo enviando directamente la imagen a un modelo visión-lenguaje de propósito general. La representación intermedia constituye una parte central del problema y debe ser observable, verificable y consultable.

Modalidad del trabajo

El TPI se desarrollará en **grupos de entre 3 y 5 integrantes** y tendrá como marco una competencia organizada por la cátedra en **Kaggle**. La competencia permitirá trabajar sobre un dataset común, enviar las predicciones producidas por cada sistema y recibir una evaluación automática. El acceso a la competencia se encuentra en este enlace de Kaggle.

Cada envío válido será evaluado por Kaggle y su resultado aparecerá en un *leaderboard*. El puntaje automático combinará un núcleo común de detección, segmentación y representación estructurada de la escena. Este ranking ofrecerá una referencia común y permitirá observar la evolución de las soluciones durante el desarrollo. Sin embargo, **la posición en el leaderboard no determinará por sí sola la calificación**: también se evaluarán la metodología, la experimentación, la calidad del análisis, la trazabilidad de la pipeline y la comprensión técnica del grupo.



Cada grupo deberá preparar:

- Una **notebook en Kaggle**, reproducible y guardada con todas sus salidas, que implemente y evalúe la solución completa.
- Los **envíos a la competencia**, con un `SceneJSON` por imagen que incluya, cuando corresponda, las respuestas obtenidas mediante los modos determinista y con interpretación por LLM.
- Un **reporte técnico en Markdown** que documente decisiones, experimentos, resultados y análisis de errores.
- Una **defensa oral**, en la que todos los integrantes deberán explicar y justificar el trabajo realizado.

La competencia será una herramienta de evaluación cuantitativa y comparación, mientras que la notebook, el reporte y la defensa permitirán evaluar el proceso completo que produjo esos resultados.

Requerimiento general

En una notebook reproducible en Kaggle, cada grupo deberá implementar una pipeline que:

1. Explore y caracterice el dataset.
2. Detecte y localice objetos de un vocabulario abierto o configurable.

3. Utilice las cajas detectadas como prompts para segmentar instancias con SAM2.
4. Construya una representación estructurada de la escena en JSON.
5. Derive de cajas y máscaras propiedades útiles de los objetos.
6. Registre una traza auditable de la ejecución de sus módulos.
7. Responda preguntas mediante un modo determinista que utilice únicamente la información estructurada.
8. Para alcanzar la promoción, genere una descripción factual mediante un LLM liviano de texto e incorpore un segundo modo en el que el LLM interprete la consulta.
9. Evalúe cuantitativamente las etapas y analice casos correctos, ambiguos y fallidos.

La arquitectura exacta, los modelos y los parámetros podrán variar entre grupos siempre que se respeten las restricciones del enunciado y las decisiones se encuentren justificadas y evaluadas.

Dataset

Se utilizará el dataset público **Comprensión visual de escenas**, construido por la cátedra a partir de imágenes y anotaciones de LVIS/COCO:

El dataset contiene:

- 120 imágenes.
- Un vocabulario evaluable de 120 categorías.
- Anotaciones de instancias, bounding boxes y máscaras compatibles con COCO/LVIS.
- Metadatos de evaluación LVIS para tratar correctamente categorías no exhaustivas y categorías verificadas como ausentes.
- Información de atribución y licencias.

Dos conjuntos complementarios

- **Dataset de desarrollo:** las 120 imágenes anotadas descriptas anteriormente. Se utilizarán para EDA, experimentación, ajuste de la pipeline y cálculo de métricas locales. De este conjunto, 20 imágenes tendrán consignas asociadas para el benchmark de preguntas y descripción.
- **Test de la competencia:** 60 imágenes nuevas y sin anotaciones públicas. Se utilizarán para evaluar detección, segmentación y representación estructurada, y para calcular el leaderboard de Kaggle.

Ambos conjuntos son disjuntos. El archivo `submission.csv` conservará exactamente las columnas `image_id,SceneJSON` y contendrá **80 filas**: una por cada una de las 60 imágenes del test de la competencia y una por cada una de las 20 imágenes de desarrollo seleccionadas para el benchmark de lenguaje, siguiendo `sample_submission.csv`.

En las 60 filas del test se evaluará la pipeline visual. En las 20 filas del benchmark, el `SceneJSON` deberá incluir además el campo `questions` y, para quienes aspiren a la promoción, `questions_llm`. Las anotaciones privadas del test sólo estarán disponibles para la evaluación de Kaggle; las anotaciones públicas de las imágenes de desarrollo no podrán utilizarse para asistir la inferencia ni para construir sus respuestas.

Las imágenes presentan distinta cantidad de instancias, tamaños de objeto, repeticiones y niveles de oclusión. Esta diversidad forma parte del problema y deberá ser considerada en el análisis.

Cómo interpretar las anotaciones LVIS

LVIS utiliza una evaluación *federada*: como su vocabulario es amplio, no se supone que todas las categorías hayan sido verificadas exhaustivamente en cada imagen. Por lo tanto, **la ausencia de una anotación no implica necesariamente que el objeto esté ausente**. Para cada imagen se distinguen los siguientes casos:

- **Categoría positiva:** tiene una o más instancias anotadas. Sus bounding boxes y máscaras constituyen el ground truth; no detectar una de esas instancias produce un falso negativo.
- **Categoría verificada como ausente** (`neg_category_ids`): fue revisada y se confirmó que no aparece en la imagen. Una predicción de esa categoría puede contarse como falso positivo.
- **Categoría positiva no exhaustiva** (`not_exhaustive_category_ids`): la categoría aparece, pero LVIS advierte que podrían existir instancias adicionales sin anotar. Las instancias anotadas se evalúan normalmente; una predicción adicional que no se asocia con ellas se ignora en lugar de contarse automáticamente como falso positivo.
- **Categoría no verificada:** no tiene instancias anotadas y tampoco figura como negativa. No puede afirmarse si está presente o ausente, de modo que sus predicciones no se utilizan para calcular falsos positivos.

> **Ejemplo.** Si una imagen contiene una instancia anotada de *horse*, declara *car* como categoría negativa y no verificó *chair*, omitir el caballo produce un falso negativo y predecir un auto produce un falso positivo. En cambio, una predicción de silla no puede considerarse falsa con esa anotación y se ignora al evaluar precisión.

El recall se calcula sobre las instancias efectivamente anotadas. La precisión sólo considera predicciones para las que LVIS permite decidir si son correctas o falsas. Por eso deberá informarse junto con su **cobertura de evaluación**: una cantidad baja de falsos positivos también podría deberse a que muchas predicciones pertenecen a categorías no verificadas.

Uso permitido de las anotaciones

Las anotaciones podrán utilizarse para:

- Realizar el análisis exploratorio.
- Construir el ground truth de evaluación.

- Calcular métricas de detección, segmentación, atributos, relaciones y preguntas.
- Seleccionar ejemplos para el análisis de errores.

No podrán utilizarse para asistir la inferencia. En particular, queda prohibido consultar las anotaciones para elegir las clases presentes, completar objetos faltantes, corregir cajas o máscaras, construir el JSON predicho o responder preguntas.

El pipeline entregado deberá poder procesar una imagen arbitraria sin acceder a sus anotaciones. La cátedra podrá verificar esta condición durante la defensa mediante imágenes no incluidas en el dataset.

Respecto del test de la competencia, tampoco se permite identificar las imágenes originales para buscar sus etiquetas en LVIS, COCO u otra fuente, reconstruir o descargar sus anotaciones, etiquetarlas manualmente ni corregir las predicciones caso por caso. Todas las salidas deberán ser producidas automáticamente por la misma pipeline documentada en la entrega.

Desarrollo del trabajo

1. Análisis exploratorio del dataset

El EDA deberá permitir comprender el problema antes de seleccionar modelos y parámetros. No se espera una colección mecánica de gráficos sino que cada visualización o número presentado deberá responder una pregunta o motivar una decisión posterior.

2. Detección open-vocabulary

Se deberá utilizar al menos un detector *open-vocabulary* (por ejemplo YOLO-World, OWLv2 u OWL-ViT, Grounding DINO, etc). Se podrán combinar.

El grupo deberá justificar, como mínimo, el detector elegido, el vocabulario utilizado, el umbral de confianza, la resolución de entrada y el criterio de supresión de duplicados.

3. Segmentación de instancias con SAM2

Las bounding boxes predichas se utilizarán como prompts para SAM2 (u otra alternativa debidamente justificada).

Dentro de la implementación, las máscaras podrán representarse mediante RLE, polígonos o referencias a archivos. Sin embargo, todo `SceneJSON` entregado en `submission.csv` deberá codificarlas como RLE no comprimido en orden column-major, utilizando los campos `size` y `counts` definidos por `scene_schema.json`. Se evitará almacenar matrices completas cuando no sea necesario.

Cada grupo deberá utilizar las máscaras para derivar propiedades, por ejemplo:

- Área y área relativa.
- Centroide.
- Posición aproximada.
- Tamaño relativo.
- Color dominante calculado dentro de la máscara.

- Superposición o proximidad entre objetos.
- Refinamiento de relaciones espaciales.

4. Representación estructurada de la escena

El resultado de percepción deberá transformarse en un JSON con objetos a nivel de instancia. El formato exacto podrá extenderse, pero deberá incluir como mínimo:

- Información de la imagen.
- Lista de objetos con identificadores únicos.
- Clase, confianza y bounding box por objeto.
- Referencia o codificación de la máscara.
- Propiedades geométricas derivadas.
- Atributos y relaciones implementados por el grupo.

Formato de entrega. La estructura completa del `SceneJSON`, los campos obligatorios y un ejemplo válido se encuentran en `scene_schema.json` y `sample_submission.csv`. Ambos archivos deberán utilizarse como referencia normativa para construir y validar la solución.

El JSON deberá ser válido y consistente para su evaluación.

La competencia utilizará esta misma representación como artefacto de envío. La cátedra publicará un núcleo mínimo mediante `scene_schema.json`, con información de imagen, objetos, cajas, máscaras RLE, centroide, área relativa, posición, tamaño y relaciones espaciales. Para las imágenes que tengan consignas asociadas, el esquema requerirá `questions`. El campo `questions_llm` será obligatorio para quienes aspiren a la promoción.

Las posiciones oficiales de la grilla de 3×3 son `top_left`, `top_center`, `top_right`, `middle_left`, `center`, `middle_right`, `bottom_left`, `bottom_center` y `bottom_right`.

5. Trazabilidad de la pipeline

La solución deberá estar modularizada y producir una traza auditable de cada ejecución.

El objetivo es poder reconstruir cómo se obtuvo un resultado y localizar en qué etapa se originó o propagó un error.

6. Preguntas y respuestas sobre la escena

El sistema deberá responder preguntas resolubles a partir del JSON. No se pretende implementar VQA abierto: las operaciones y atributos soportados deberán encontrarse explícitamente definidos.

Para que esta etapa resulte auditable sin convertirla en un segundo dataset masivo, la cátedra publicará un benchmark fijo con consignas asociadas a **20 imágenes seleccionadas entre las 120 imágenes de desarrollo**. Para cada una habrá una consigna de descripción y dos preguntas cerradas. El conjunto contendrá:

- 20 descripciones breves, una por imagen.
- 10 preguntas de conteo y 10 de existencia.
- 5 preguntas de posición y 5 de tamaño relativo.
- 10 preguntas de relaciones espaciales.

Las preguntas cerradas tendrán una respuesta normalizada: entero para conteo; `yes`/`no` para existencia y relaciones; una de nueve regiones para posición; y `small`, `medium` o `large` para tamaño. Si una consulta no puede resolverse con el JSON predicho, deberá informarse explícitamente como `unsupported` o información insuficiente, en lugar de inventar una respuesta.

El benchmark contempla dos modos, asociados a distintos niveles de logro:

- **Modo determinista, requerido para aprobado:** además del texto de la pregunta, recibirá los campos estructurados provistos por la cátedra, como el tipo de operación, el objeto o los objetos involucrados y el tipo de salida esperada. Con esa información deberá construir y ejecutar directamente una consulta sobre el `SceneJSON`, sin utilizar un LLM para interpretar la pregunta. Sus resultados se incluirán en `questions`.
- **Modo con LLM, requerido para promoción:** el módulo de interpretación recibirá únicamente el texto de la pregunta y deberá producir un JSON intermedio con la intención, los objetos o filtros relevantes y el tipo de salida. Ese JSON será validado y ejecutado por el mismo motor determinista sobre el `SceneJSON`. Las respuestas a las preguntas y las consignas de descripción se incluirán en `questions_llm`.

En ambos modos, la única fuente de información visual será el `SceneJSON` predicho. Cuando se implemente el modo con LLM, el modelo no podrá acceder a los campos estructurados de la pregunta reservados para el modo determinista, ni reemplazar al motor de ejecución, calcular la respuesta por su cuenta o inventar información faltante. Los conteos, comparaciones y filtros finales deberán resolverse mediante código.

La arquitectura común será:

$$\text{Entrada de consulta} \rightarrow \text{JSON intermedio validado} \rightarrow \text{ejecución determinista} \rightarrow \text{respuesta normalizada y verbalización}$$

El sistema deberá manejar preguntas ambiguas, atributos no disponibles y consultas fuera de alcance. Una respuesta incompleta pero sustentada es preferible a una respuesta fluida inventada.

Cada elemento de `questions` y, cuando corresponda, de `questions_llm` deberá conservar `question_id`, `type` y `question`, e incluir `answer`, `normalized_answer`,

`evidence` y `confidence`. Los elementos de `questions_llm` deberán incluir además, en `interpreted_query`, el JSON intermedio generado por el LLM. El campo `evidence` referenciará los identificadores de los objetos del mismo `SceneJSON` que sustentan la respuesta. En las imágenes sin consignas asociadas, ambos campos podrán omitirse o presentarse como listas vacías. Para el nivel aprobado, `questions_llm` podrá omitirse o presentarse como una lista vacía.

7. Evaluación y experimentación

La evaluación deberá considerar cada etapa y también el comportamiento end-to-end. Cada grupo deberá definir las métricas apropiadas para cada caso.

Detección

Segmentación

Representación estructurada

Preguntas y descripción

Proceso experimental

El grupo deberá definir una línea base y realizar al menos una comparación controlada con una alternativa concreta de mejora.

Se deberán registrar configuración, métricas, ejemplos y conclusiones. También deberán documentarse intentos que no hayan mejorado el sistema cuando resulten relevantes para comprender el proceso.

Criterios de evaluación y niveles de logro

> Esta sección se utilizará como **checklist de corrección**. Cada criterio se considerará cumplido o no cumplido a partir de la notebook, el reporte y la defensa oral. Los niveles son acumulativos: para alcanzar uno deberán cumplirse todos sus criterios y los de los niveles anteriores.

El leaderboard utilizará un *PipelineScore* compuesto por 40 % de F_1 de cajas, 35 % de F_1 de máscaras, 15 % de consistencia de la representación estructurada y 10 % de evaluación del benchmark de preguntas. El componente de preguntas se calculará sobre las imágenes de desarrollo que tengan consignas asociadas y promediará los modos determinista y con LLM. Los umbrales de F_1 de detección indicados a continuación deberán calcularse además sobre el conjunto anotado de desarrollo, con IoU $\geq 0{,}5$ y respetando la política de anotación LVIS. Alcanzar un valor cuantitativo no compensa una pipeline incompleta ni un análisis insuficiente.

Condiciones de validez

Todas son obligatorias:

- Notebook reproducible, entrega completa y resultados auditables.
- Inferencia sin consultar ni utilizar las anotaciones del dataset.

- Reporte consistente con la implementación.
- Defensa oral con comprensión técnica de todos los integrantes.

Aprobado

- EDA relevante que caracterice el dataset, identifique sus principales dificultades y fundamente decisiones posteriores.
- Pipeline visual completa: detección, SAM2 y construcción del JSON de escena.
- F_1 de detección de instancias igual o superior a 0,40.
- Uso efectivo de las máscaras y representación JSON válida.
- Inclusión válida del campo `questions` en los `SceneJSON` correspondientes, obtenido en modo determinista a partir de la pregunta y sus campos estructurados, con soporte efectivo para existencia y conteo.
- Ejecución modular, reproducible y con una traza básica de etapas, tiempos y estados.
- Evaluación cuantitativa con métricas seleccionadas y justificadas por el grupo, incluida una evaluación básica de la fidelidad de las respuestas deterministas respecto del JSON.
- Comparación experimental básica y análisis de casos correctos y fallidos.

Promoción

Además de lo requerido para aprobado:

- F_1 de detección de instancias igual o superior a 0,50.
- Atributos y relaciones útiles, accesibles mediante consultas deterministas sobre el JSON.
- Inclusión válida del campo `questions_llm` en los `SceneJSON` correspondientes, obtenido mediante un LLM liviano y ejecutable en Kaggle que reciba únicamente el texto de la pregunta y genere el JSON intermedio.
- Ambos modos capaces de resolver preguntas de ubicación, comparación, atributos o relaciones; en el modo con LLM, el modelo deberá transformarlas en un JSON intermedio válido y verbalizar el resultado sin reemplazar al motor determinista.
- Manejo explícito de preguntas ambiguas, no soportadas o sin información suficiente.
- Tracing completo que permita reconstruir la ejecución, sus parámetros, artefactos y errores, y utilizar las trazas para evaluar las etapas y analizar la propagación de errores.
- Mejora justificada mediante experimentación controlada y comparación contra una línea base.

Sobresaliente

Además de lo requerido para promoción:

- Una extensión relevante y no trivial del sistema.
- Una motivación o hipótesis clara para esa extensión.
- Evaluación cuantitativa de su aporte.
- Análisis crítico que demuestre qué mejora, qué limitaciones conserva y qué costo introduce.

La extensión podrá abordar cualquier componente del trabajo. Acumular funcionalidades sin demostrar su aporte no garantiza el nivel sobresaliente.

Entregables

- Reporte técnico en **Markdown (.md)**, breve, técnico y enfocado en decisiones, experimentos y resultados.
- Notebook de Kaggle con el desarrollo completo y todas las salidas guardadas.
- Archivo `submission.csv` válido, con 80 filas: 60 correspondientes al test de la competencia y 20 a las imágenes de desarrollo del benchmark de lenguaje. Estas últimas deberán integrar las respuestas deterministas en `questions`; para quienes aspiren a la promoción, deberán incluir además las respuestas y descripciones generadas mediante LLM en `questions_llm`.

No se aceptarán reportes en PDF, DOCX, Google Docs exportado u otro formato distinto de Markdown, salvo indicación posterior de la cátedra.

Requisitos de la notebook

- Debe ejecutarse completamente en Kaggle sin archivos externos distintos del dataset oficial o de recursos públicos descargados por la propia notebook.
- Debe fijar semillas cuando exista aleatoriedad y registrar las configuraciones relevantes.
- Debe estar organizada mediante secciones y explicaciones en Markdown.
- Debe conservar todas las salidas necesarias para revisar el trabajo sin volver a ejecutar.
- Debe distinguir claramente ground truth, predicciones y datos derivados.
- Debe evitar código o comentarios redundantes que no puedan ser explicados por el grupo.
- Debe guardar resultados cuantitativos en formatos auditables.
- El reporte y la notebook deben corresponderse exactamente.

- Cada integrante debe poder explicar el código y las decisiones durante la defensa.
- Nombre requerido: `TPI_IA_2026P_[nro_de_grupo]`.
- No podrá modificarse luego de la fecha de entrega.

Estructura sugerida del reporte técnico

1. **Portada**.
2. **Análisis del dataset**.
3. **Metodología**.
4. **Resultados**.
5. **Discusión y análisis de errores**.
6. **Conclusiones y trabajo futuro**.

Cronograma

- Fecha límite de entrega: **1 de noviembre de 2026 a las 23:59 hs**.
- Defensas orales: **A definir**.

Reglas

- El trabajo se realizará en grupos de mínimo 3 personas y máximo 5 personas.
- La notebook deberá ser reproducible y ejecutable en Kaggle.
- No se permite utilizar las anotaciones para asistir o corregir la inferencia.
- No se permite identificar, buscar anotaciones ni etiquetar manualmente las imágenes del test de la competencia.
- No se permite enviar la imagen original al LLM de descripción o preguntas.
- Deberán declararse los modelos, recursos externos y código reutilizado.
- Toda manipulación artificial de resultados o métricas será motivo de desaprobación.
- No se permite modificar la entrega luego del vencimiento.

Presentación oral

La presentación será una instancia de defensa y justificación técnica de no más de 10 minutos, seguida de preguntas de la cátedra.

Todos los integrantes deberán demostrar comprensión del trabajo completo.